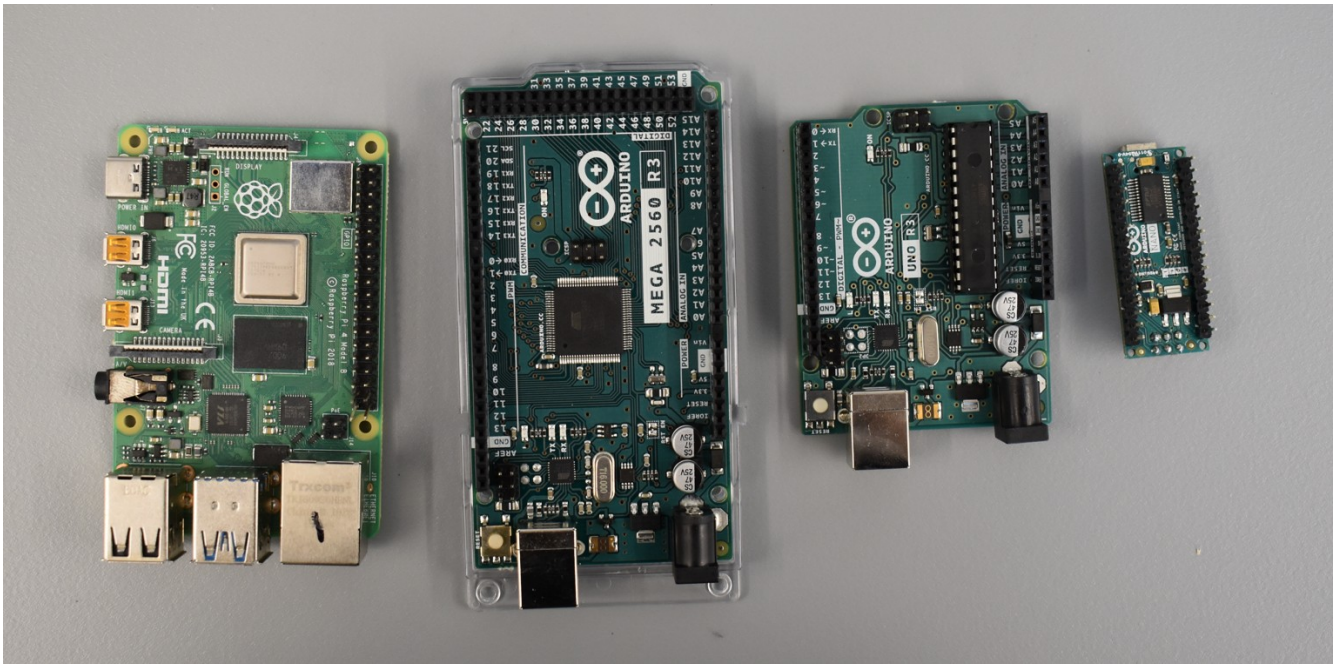
A Crash Course in Arduino Programming
Lesson 1

A. Introduction

Welcome to the Arduino Programming Crash Course! This is the first in a series of lessons aimed to set you up for programming success. You'll need your laptop and an Arduino if you would like to follow along, and, as always, make sure you bring a can-do-attitude!

B. Little Computers

Our lab hosts a variety of magic, green rectangles. Along with this handout, you should have received the **pinout diagrams** for the Arduino Uno, Arduino Mega, Arduino Nano, and Raspberry Pi 3[1]. Take some time to familiarize yourself with each. How are they similar? How are they different?



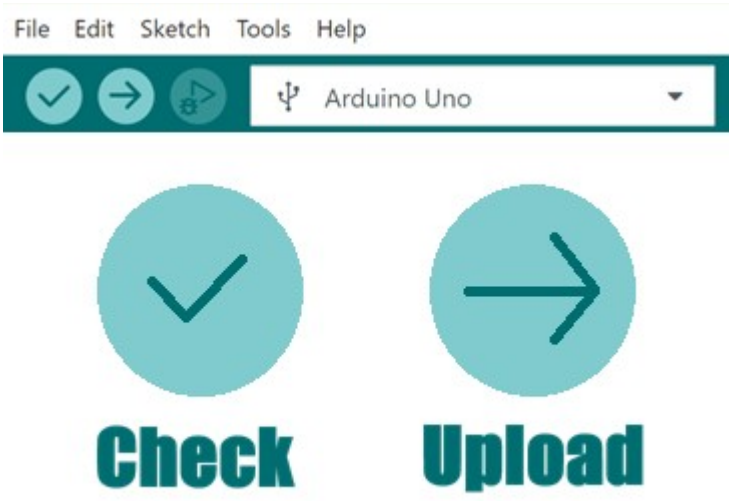
The Arduino Uno, Mega, and Nano are **microcontrollers (MCUs)**. In contrast, the Raspberry Pi 3 is a **microprocessor (MPU)**. Typically, an MCU is a computer on a *single* **integrated circuit**. MCUs are commonly used for direct control and communication with hardware. For instance, you might use an MCU to spin the wheels on a robot, change the colors on a light display, or store sensor data to an SD card[2]. On the other hand, MPUs are better suited for tasks that require higher-level processing or computation. MPUs are useful for computer vision, machine learning, and tasks that require more memory than the average MCU can provide.

C. How Computers Read Instructions

You have likely had some experience writing instructions for some form of Arduino. Additionally, you have likely been using an **integrated development environment** called the **Arduino IDE**. The **programming language** used here is **C++**[3].

---

1 These are far from the only boards you'll run into, but they are some of the most commonly used in AU[2] projects.
2 If we want to sound fancy, we can say MCUs are commonly applied to *embedded systems*.
3 Kind of, sort of, depends on who you ask. We can do a deeper dive into this as an advanced-topics lesson if requested.



The instructions given directly to a target machine are not human-interpretable. A **compiler** takes instructions written in a human-interpretable programming language and translates it into the language of the machine. After the code is compiled, it can be **executed** to perform a task. The **check** button compiles the code. The **upload** button compiles the code *and* uploads it to the Arduino where it is executed.

Arduino programs (and most programs in general) are executed from top to bottom. In the Arduino IDE, there is a **setup** and a **loop**. Code in the setup is executed *first*, and it is executed *only once*. Code in the loop is executed *after* the setup, and it is executed until the program is terminated. Things created in the setup are not necessarily accessible in the loop; this has to do with **scope** which is something we will address in a later lesson[4].

D. Dealing with Bad Instructions

Often, we write instructions, but it turns out our instructions suck. This makes the Arduino mad, so it yells at us in angry red text. Let's go over some common error messages and their causes.

1. Where's that Board?

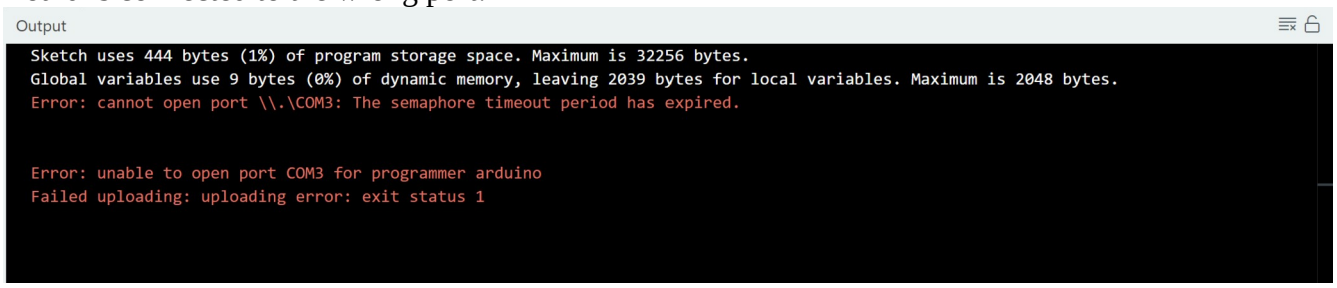
These are some of the first errors any Arduino user encounters. They occur if the board is unplugged or connected to the wrong port.

Board is unplugged:

```
Sketch uses 444 bytes (1%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Maximum is 2048 bytes.
Error: cannot open port \\.\COM17: The system cannot find the file specified.


Error: unable to open port COM17 for programmer arduino
Failed uploading: uploading error: exit status 1
```

4 You could say this is… out of the "scope" of this lesson!

Board is connected to the wrong port:



2. You Said a Bad Word

When you name things in your program, you gotta follow some rules or else your program will explode[5].

The Commandments of Naming:

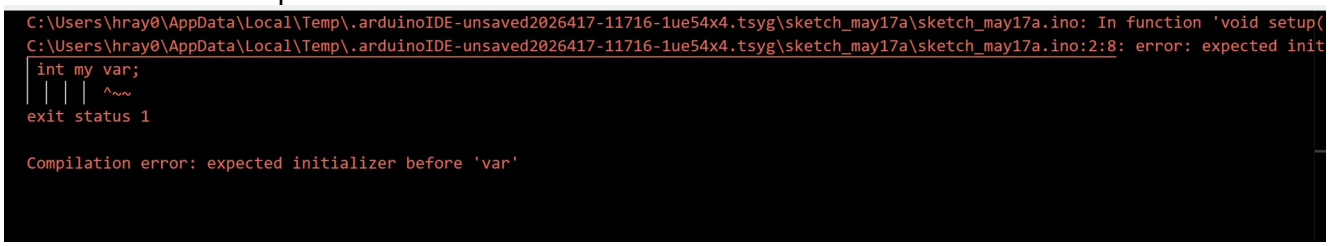
1. You must not start a name with a number.
2. The name shall not contain spaces or special characters.
3. You may not use names that Arduino has claimed as "keywords."

A **keyword** is a word that already does something special such as if, else, void, and byte. These words are typically, but not always, highlighted in the code by the Arduino IDE when you type it.

Acceptable name examples: myVariable, yourMom, goPhysics, silly_goose, myNum123, abcdefg
Forbidden name examples: if, 123, two words, #goPhysics, PIN1

Variable name starts with a number:

Variable name has a space in it:



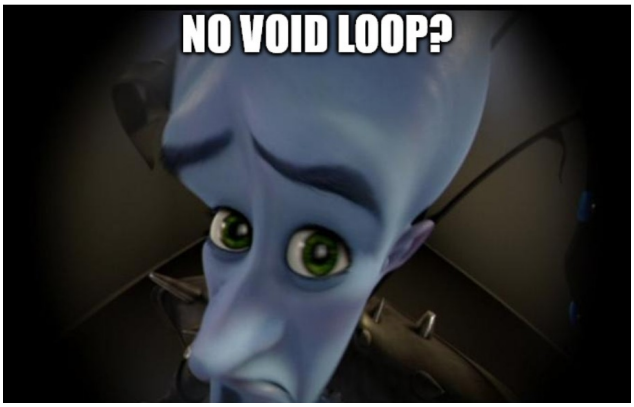
5 There are also ways to name variables that won't make your code explode but will make people judge you. These variable names are not breaking rules, but they are breaking *conventions*.

Tried to name variable after a keyword:

```
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino: In function 'void setup()':
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino:2:5: error: expected unqualified-id before 'if'
 int if = 3;
     ^~
exit status 1

Compilation error: expected unqualified-id before 'if'
```

3. The Default Sketch Was Like That For a Reason

Arduino does not care if there is nothing in your void loop. If you do not have both the setup and the loop in your program, it will not compile.



No loop:

```
C:\Users\hray0\AppData\Local\Temp\ccgPJAhH.ltrans0.ltrans.o: In function `main':
C:\Users\hray0\AppData\Local\Arduino15\packages\arduino\hardware\avr\1.8.7\cores\arduino/main.cpp:46: undefined reference to `loop'
collect2.exe: error: ld returned 1 exit status
exit status 1

Compilation error: exit status 1
```

No setup:

```
C:\Users\hray0\AppData\Local\Temp\cc4H4CdJ.ltrans0.ltrans.o: In function `main':
C:\Users\hray0\AppData\Local\Arduino15\packages\arduino\hardware\avr\1.8.7\cores\arduino/main.cpp:43: undefined reference to `setup'
collect2.exe: error: ld returned 1 exit status
exit status 1

Compilation error: exit status 1
```

4. The Classic;

No Semicolon:

```
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino: In function 'void setup()
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino:3:1: error: expected ',' o
 }
 ^
exit status 1

Compilation error: expected ',' or ';' before '}' token
```

One thing that might seem confusing when you encounter this error is the fact that the IDE tends to show a red highlighted line *after* the line where the error occurred.

```
void setup() {
int urMom = 0
}
```

Arduino does not actually care if the semicolon is on the *same* line as “int urMom = 0”. It just cares that there is a semicolon *anywhere after* your line of code and *before* whatever needs to be read next. So, the line with the “}” is the actual location of the error. While against convention, the next piece of example code will compile.

```
void setup() {
int urMom = 0
;}
```

5. There is No Spoon

Sometimes you ask the Arduino to do something with a variable, but you never made that variable.

Undeclared variable:

```
void setup() {
Serial.begin(9600);
}

void loop() {
  // instead, only try to realize the truth...
  Serial.print(spoon);
}
```

```
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino: In function 'void loop()'
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino:7:16: error: 'spoon' was n
  Serial.print(spoon);
               ^~~~~
exit status 1

Compilation error: 'spoon' was not declared in this scope
```

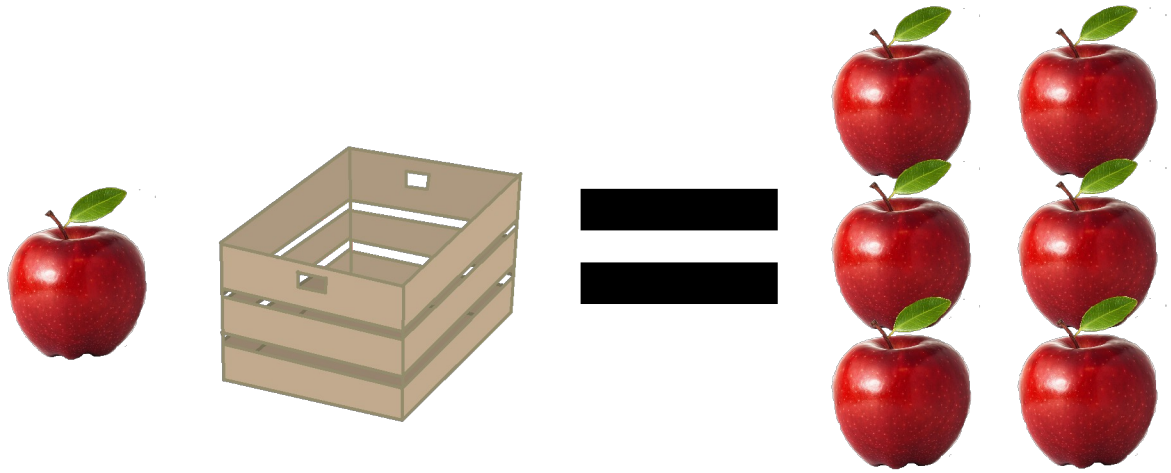
Declared, but not in this scope:

```
void setup() {
Serial.begin(9600);
String spoon = "There is no spoon";
}

void loop() {
  // instead, only try to realize the truth...
  Serial.print(spoon);
}
```

```
Output
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino: In function 'void loop()'
C:\Users\hray0\AppData\Local\Temp\.arduinoIDE-unsaved2026417-11716-1ue54x4.tsyg\sketch_may17a\sketch_may17a.ino:8:16: error: 'spoon' was n
  Serial.print(spoon);
               ^~~~~
exit status 1

Compilation error: 'spoon' was not declared in this scope
```

E. Variables and Data Types

We mentioned **variables** a lot in the previous section, but what is a variable anyway? A variable is something that stores data of a specific type. Imagine a wooden crate filled with apples. The crate can *only* hold apples, so the data type is “apple.” We identify the container of apples by the name “wooden crate.” This is the variable’s name. If you put 6 apples in the wooden crate, then it has a value of 6. We could say “`apple woodenCrate = 6;`”.



I told you that a variable stores a specific type of data, but what does that really mean? A variable’s **datatype** determines what the variable can do, the range of values it can represent, and how those values are represented in memory. Most of the datatypes we work with are **numeric**, such as integers,

floats, longs, and doubles. Strings and characters let you store the letters and symbols found on your keyboard. Booleans are variables that can store either the value **true** or the value **false**. There are many more datatypes to explore, but for today, we'll focus on a handful that you will be encountering frequently.

1. bool

The boolean is a datatype that can have one of two values: true or false. It is more of a *logical datatype* than a *numeric datatype* but it can be represented by 0s and 1s. True is usually associated with digital HIGH, and false is usually associated with digital LOW. We'll be using this datatype a lot when we talk about **conditional statements**.

2. int

Integers can be **signed** or **unsigned**. Unless specified, integers are signed by default; this means that the variable can have a positive or negative value. Unsigned integers can only be positive. On the Arduino Uno, integers are 16-bits (two bytes) and can have values ranging from -32,768 to 32,767. Integers are a good choice for variables used for counting, values that do not require decimal precision, and numbers that don't get very big. For example, you could use an integer to count the number of times a button is pressed, to indicate how many LEDs you want to turn on, or to keep track of how many times a loop has been executed.

3. float

Floats are 32 bit (4 byte) numbers with decimal points. They can range in value from -3.4028235E+38 to 3.4028235E+38. We typically use floats when carrying out calculations that have a division operation and when we need decimal precision.

4. Byte

A **byte** is a unit consisting of 8 **bits**. The bit is the fundamental unit of information in computing. Similar to the boolean, the bit can have one of two possible values: 0 or 1. Numbers that represented with 0s and 1s are **binary** numbers. We'll see later that bytes and booleans can have similar operations performed on them. Bytes are often used when you need to represent a small amount of information, such as when you make a variable that contains an address in I2C. You may also use the byte datatype when you are **bit-banging**.

F. Operations

We don't just want to create variables, we want to do something with them. This is where operations come in.

1. To = or to ==

What's the difference between "=" and "==". The = sign is how we assign a value to a variable.

```
<datatype> <variable name> = <variable value>;
```

When we use == we are checking to see if two values are equivalent. Let's say we have two values, `A` and `B`. We perform the following operation.

```
bool isEqual = (A == B);
```

If `isEqual` is *true*, then `A` and `B` are equivalent. If `isEqual` is *false*, `A` and `B` are not equivalent.

2. Math

You are already familiar with **multiplication (*)**, **division (/)**, **addition (+)**, and **subtraction (-)**, but there are still some traps you can fall into here. The first trap is the order of operations. The code does not necessarily perform every operation in the order it appears from left to right. 3+3/6 will not give you the same number as (3+3)/6 (TRY IT!). 3+3/6 is interpreted as 3 + (3/6), so you'll get 3.5. (3+3)/6 gives you 1. You could learn and memorize the order of operations for C++, and while it's a useful thing to know, I'm not going to encourage you to do that. Instead, you should just use parentheses to indicate your desired order of operations. DO NOT TRUST YOUR MEMORY. USE PARENTHESES.

Your second trap of the day is integer division. Let's use an example:

```
int myNum = 5/2;
```

What's your answer? If we were doing the operation in our heads, we would get 2.5. However, integers don't have decimal places. You might guess that `myNum` would take on a value of 3, and that's a reasonable guess. Unfortunately, that is also an incorrect guess. Integers do not round, they **truncate**. Truncation means we just cut off the decimal portion of the number, so my `myNum` has a value of 2.

Additionally, care must be taken when mixing numerical data types in operations. If performing an operation between an integer and a float, Arduino will make the result a float unless told to do otherwise (TRY IT!).

In terms of syntax, you've probably seen operations performed like this:

```
int myNum = 1;
myNum = myNum + 5;
```

For brevity, you can also perform this operation like this:

```
int myNum = 1;
myNum += 5;
```

3. Modulo

There is one common math operation that you may not be familiar with. It's called **modulo** and it is represented by the % symbol. Modulo is like division, except it returns the remainder. For example:

```
int myNum = 5%2;
```

The value of `myNum` is 1.

4. Increment and Decrement

If you want to add or subtract 1 from a number, you can use **increment** or **decrement**. Here is an example of the syntax:

```
int myNum = 1;  // make an integer with a value of 1
myNum++;        // add 1, myNum now equals 2
myNum--;        // subtract 1, myNum now equals 1
```

These operations are common in loops that use **counters**.